

ChainGuard Lite

Two-Day Antigravity Build Specification for ETHOnline 2026

Document purpose: This document is the complete implementation brief for Antigravity or another AI coding agent. It defines exactly what to build, what not to build, how to build it phase by phase, and how to verify that the final project qualifies for the selected hackathon tracks.

Target build time: Two days.

Primary tracks:

1. The Graph — Best AI Tooling or AI Use Case with The Graph (From Scratch).
2. Hedera — AI & Agentic Payments on Hedera.

Optional track only if the main project is complete: Bazantic — Help an Agent Use Your Hackathon Project.

1. Product definition

1.1 Product name

ChainGuard Lite

1.2 One-sentence description

ChainGuard Lite is a testnet-only AI service that retrieves live blockchain activity through The Graph, creates a transparent wallet or protocol safety screening report, and delivers the report only after an AI agent completes a small Hedera testnet x402 payment.

1.3 Problem

Crypto users often cannot quickly interpret wallet activity, protocol interactions, transaction frequency, or unusual activity before interacting with an unfamiliar address or protocol. The data exists on-chain, but it is difficult for ordinary users and autonomous agents to query and understand.

1.4 Solution

ChainGuard Lite provides one simple workflow:

1. The user enters a testnet wallet address.
2. The agent requests a safety report.
3. The report API returns HTTP 402 Payment Required.
4. The agent signs a small testnet HBAR payment.
5. Hedera x402 verifies or settles the payment.
6. The service queries live data from The Graph.
7. The service calculates transparent signals.
8. An AI model converts the evidence into a concise report.
9. The frontend displays the report, payment status, evidence, limitations, and testnet transaction information.

1.5 Important product limitation

This is an educational screening tool, not a scam detector, compliance system, investment adviser, or guarantee of transaction safety.

The application must use the following wording throughout the interface:

This report is an educational screening result based on indexed testnet data. It is not financial advice and does not guarantee that an address or transaction is safe.

2. Non-negotiable scope rules

Antigravity must follow these rules exactly.

2.1 Required

- Use live data from The Graph.
- Use Hedera testnet for the x402 payment flow.
- Make at least one real paid request end to end.
- Use a dedicated low-balance testnet payer account.
- Keep the code open-source and easy to run.
- Provide a simple browser interface.

- Provide a CLI or script that can reproduce the paid request.
- Include clear logs for payment and data retrieval.
- Include a transparent scoring rubric.
- Include error states and provider fallbacks.

2.2 Forbidden for this two-day version

Do not build any of the following:

- Mainnet support.
- Real fund transfers.
- Trading, swaps, approvals, or portfolio rebalancing.
- Smart-contract deployment unless absolutely required by an existing x402 setup.
- User accounts or authentication.
- Database persistence.
- Social login.
- Mobile application.
- Token or NFT.
- DAO or governance.
- Multi-chain support.
- Custom marketplace.
- Custom Substreams pipeline.
- Complex wallet connection.
- Custom scam-label database.
- Claims that the tool can prove an address is malicious.

If a feature is not required for the two selected tracks, do not build it.

3. Track qualification strategy

3.1 The Graph track

The Graph is a load-bearing part of the project because the report cannot be generated

without live indexed blockchain data.

The implementation must demonstrate:

- A live GraphQL request to a Graph provider or The Graph Subgraph MCP.
- Retrieval of structured blockchain entities or events.
- Transformation of raw data into evidence fields.
- AI reasoning over those evidence fields.
- A final recommendation that is more useful than simply displaying raw query results.

The README must include:

- The exact Graph query or MCP request.
- The provider configuration.
- The returned fields used by the report.
- A sample response.
- An explanation of why the report depends on The Graph.

3.2 Hedera x402 track

Hedera is a load-bearing part of the project because the report API is paid through Hedera testnet x402.

The implementation must demonstrate:

- The report API returning HTTP 402 when payment is missing.
- The agent reading the payment requirement.
- The agent signing a small testnet HBAR payment.
- The payment being verified or settled through an x402 facilitator.
- The report being returned only after payment processing.
- Testnet transaction or facilitator evidence visible in the demo.

The README must include:

- The x402 payment flow.
- The selected testnet network.
- The payment asset and amount.
- The facilitator URL or SDK configuration.
- The payer and receiver roles.

- A screenshot or log of the successful paid request.

3.3 Optional Bazantic track

Only attempt this after all required functionality is complete.

The optional integration is:

- Register the report API as an x402 gateway in Bazantic.
- Create a Recipe explaining when an agent should use the report.
- Run the same prompt once without the Recipe and once with the Recipe.
- Record whether tool selection or report quality improves.

If this takes more than two hours, stop and return to testing, documentation, and demo preparation.

4. Recommended technology stack

Use a small TypeScript monorepo or a single TypeScript application.

Layer	Recommendation
Language	TypeScript
Runtime	Node.js 20 or newer
API server	Express
Frontend	Vite + React, or static HTML if React setup becomes slow
Styling	Plain CSS or a small utility stylesheet
Blockchain payment	Hedera x402 SDK and facilitator-compatible client
Blockchain data	The Graph GraphQL endpoint; use Subgraph MCP only if it is faster to integrate
AI	Existing OpenAI-compatible endpoint or project-provided LLM helper
Validation	Zod or simple TypeScript validation
Tests	Vitest or Node test scripts

Deployment

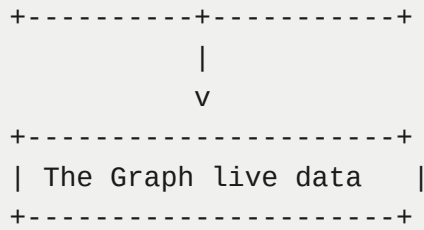
Localhost for the demo, or one simple public deployment if already available

Use the official Hedera x402 proof of concept as a reference for package names and payment flow. Do not blindly copy its mainnet functionality. Remove all mainnet paths from this project.

5. Required architecture

Plain Text

```
+-----+
|   Browser UI   |
| address + report view |
+-----+
|
| POST /api/report
v
+-----+
|   Agent server   |
| x402 payment client |
+-----+
|
| request without payment
v
+-----+
| Report API service |
| POST /v1/safety-report|
+-----+
|
| HTTP 402 if unpaid
|
| signed testnet payment
v
+-----+
| Hedera x402 facilitator|
+-----+
|
| payment verified/settled
v
+-----+
|   Report engine   |
| Graph + rules + AI |
+-----+
```



5.1 Request flow

1. Browser calls `POST /api/report` on the agent server.
2. Agent server sends a request to `POST /v1/safety-report`.
3. Report API returns HTTP 402 if no payment header exists.
4. Agent client creates and signs the Hedera testnet payment.
5. Agent client retries the request with the x402 payment header.
6. Report API verifies the payment.
7. Report engine queries The Graph.
8. Report engine calculates evidence and risk signals.
9. Report engine calls the LLM with structured evidence.
10. Report API returns the final report.
11. Agent server forwards the report to the browser.

5.2 Keep responsibilities separate

- `graphClient` : retrieves and normalizes live data.
- `riskEngine` : calculates deterministic signals.
- `reportGenerator` : creates constrained AI output.
- `paymentServer` : protects the report endpoint.
- `paymentClient` : handles the x402 retry flow.
- `web` : displays the workflow and result.

6. Repository structure

Use this structure unless the selected starter template requires a small adjustment.

Plain Text

```
chainguard-lite/
├── apps/
│   ├── agent/
│   │   └── src/
│   │       ├── server.ts
│   │       ├── payment-client.ts
│   │       └── routes.ts
│   ├── report-service/
│   │   └── src/
│   │       ├── server.ts
│   │       ├── payment-server.ts
│   │       └── routes.ts
│   └── web/
│       ├── index.html
│       └── src/
│           ├── App.tsx
│           ├── api.ts
│           └── styles.css
├── packages/
│   ├── graph-client/
│   │   └── src/index.ts
│   ├── risk-engine/
│   │   └── src/index.ts
│   ├── report-generator/
│   │   └── src/index.ts
│   └── shared/
│       └── src/types.ts
├── scripts/
│   ├── test-paid-request.ts
│   └── check-providers.ts
├── docs/
│   ├── ARCHITECTURE.md
│   └── PROVIDER_CHECK.md
├── .env.example
├── README.md
├── package.json
└── tsconfig.json
```

A simpler single-app structure is acceptable if it saves time. The separation between payment, data, scoring, and UI is more important than the exact folders.

7. Environment configuration

Create `.env.example`. Never commit real credentials.

Plain Text

```
# Application
NODE_ENV=development
AGENT_PORT=3001
REPORT_SERVICE_PORT=4021
WEB_PORT=5173

# The Graph
GRAPH_API_URL=https://YOUR_GRAPH_PROVIDER_ENDPOINT
GRAPH_API_KEY=YOUR_GRAPH_API_KEY
GRAPH_SUBGRAPH_ID=YOUR_SUBGRAPH_ID
GRAPH_NETWORK=testnet

# Hedera testnet payer
HEDERA_NETWORK=testnet
HEDERA_PAYER_ACCOUNT_ID=0.0.REPLACE_ME
HEDERA_PAYER_PRIVATE_KEY=REPLACE_ME
HEDERA_RECEIVER_ACCOUNT_ID=0.0.REPLACE_ME

# x402
X402_PAYMENT_ASSET=HBAR
X402_PRICE=0.001
X402_FACILITATOR_URL=REPLACE_ME

# AI
LLM_API_BASE_URL=REPLACE_ME
LLM_API_KEY=REPLACE_ME
LLM_MODEL=REPLACE_ME

# Safety
DEMO_ONLY_TESTNET=true
```

The application must refuse to start if `DEMO_ONLY_TESTNET` is not `true` .

The application must also reject environment values containing obvious mainnet configuration such as `mainnet` in network fields.

8. Data model and API contract

8.1 Request schema

POST /v1/safety-report

JSON

```
{
  "address": "0x0000000000000000000000000000000000000000",
  "network": "testnet",
  "subjectType": "wallet"
}
```

Allowed values:

- network : only testnet .
- subjectType : wallet or protocol .
- address : validate as a hexadecimal EVM address.

8.2 Paid response schema

JSON

```
{
  "reportId": "demo-2026-001",
  "subject": {
    "address": "0x...",
    "network": "testnet",
    "type": "wallet"
  },
  "riskLabel": "Needs review",
  "confidence": "limited",
  "summary": "The address has recent indexed activity, but the available data i",
  "reasons": [
    "Recent activity was detected in the selected data source.",
    "The indexed history contains multiple counterparties.",
    "The available evidence is limited to the selected Subgraph schema."
  ],
  "evidence": {
    "eventCount": 12,
    "uniqueCounterparties": 7,
    "recentEventCount": 5,
    "firstSeen": "2026-09-01T10:00:00Z",
    "lastSeen": "2026-09-08T10:00:00Z",
    "dataSource": "The Graph",
    "queriedAt": "2026-09-08T10:00:00Z"
  },
  "recommendedChecks": [
    "Verify the official contract address before interacting.",
    "Avoid unlimited token approvals.",
    "Test with a small amount first."
  ],
}
```

```
"limitations": [
  "This is not a scam determination.",
  "Indexed blockchain data may be incomplete or delayed."
],
"payment": {
  "network": "hedera:testnet",
  "asset": "HBAR",
  "amount": "0.001",
  "status": "verified"
}
}
```

8.3 Agent endpoint

POST /api/report

The browser calls this endpoint. The endpoint internally handles the x402 retry flow and returns the paid report to the browser.

Request:

JSON

```
{
  "address": "0x...",
  "subjectType": "wallet"
}
```

Response:

JSON

```
{
  "status": "success",
  "paymentSteps": [
    "payment_required",
    "payment_signed",
    "payment_submitted",
    "payment_verified",
    "report_generated"
  ],
  "report": {}
}
```

9. Report logic

9.1 Data normalization

The Graph response must be converted into a stable internal object before any AI call.

Plain Text

```
export type NormalizedEvidence = {  
  address: string;  
  eventCount: number;  
  uniqueCounterparties: number;  
  recentEventCount: number;  
  firstSeen: string | null;  
  lastSeen: string | null;  
  protocolInteractions: number;  
  dataComplete: boolean;  
  source: "The Graph";  
  queriedAt: string;  
};
```

9.2 Deterministic scoring

Use a simple transparent score. The exact thresholds may be adjusted after inspecting the available data.

Condition	Score
Fewer than two indexed events	+1 concern
Recent activity burst	+1 concern
More than five unique counterparties in the demo window	+1 concern
Repeated interaction with the selected protocol	-1 concern
Missing required data	Force Needs review

Risk labels:

- Low signal : score less than or equal to 0 and data is complete.

- Needs review : score 1 or incomplete data.
- High concern : score 2 or greater.

Use the label **High concern**, not **Scam** or **Malicious**.

9.3 AI prompt

The LLM must not receive private keys, payment headers, or raw secrets.

Use a prompt similar to this:

Plain Text

You are ChainGuard Lite, an educational crypto safety screening assistant.

Use only the structured evidence provided below. Do not invent facts.

Do not call the subject malicious or safe with certainty.

Do not provide investment advice.

Explain what the evidence suggests and what it cannot prove.

Return valid JSON matching the requested schema.

Required output fields:

- summary: one short paragraph
- reasons: exactly 2 or 3 evidence-based reasons
- recommendedChecks: exactly 2 or 3 practical checks
- limitations: exactly 2 limitations

Structured evidence:

{{EVIDENCE_JSON}}

Deterministic label:

{{RISK_LABEL}}

If the LLM is unavailable, return a deterministic template report. The app must remain demoable without the LLM service.

10. Two-day implementation process

Day 1 — Core integration and end-to-end functionality

Phase 1: Project setup and provider validation

Time budget: 2 hours.

Tasks

1. Create the repository.
2. Initialize TypeScript and package management.
3. Install Express, validation, GraphQL, Hedera x402 dependencies, and frontend dependencies.
4. Create `.env.example` .
5. Create `scripts/check-providers.ts` .
6. Verify that `DEMO_ONLY_TESTNET=true` is enforced.
7. Create or configure the two Hedera testnet accounts:
 - payer account;
 - receiver account.
8. Fund the payer with a small amount of testnet HBAR.
9. Verify that the selected x402 facilitator supports the chosen Hedera testnet payment path.
10. Verify one live Graph query manually.

Acceptance criteria

- `npm install` works.
- `npm run check:providers` prints a clear result for The Graph and Hedera.
- No mainnet URL or credential is required.
- A live Graph response is saved as a local development fixture only for debugging.
- A testnet payment configuration is documented in `docs/PROVIDER_CHECK.md` .

Antigravity instruction

Do not proceed to UI work until the provider check script confirms a valid Graph response and a valid Hedera testnet configuration. If the Subgraph MCP path is unstable, use a direct GraphQL request to a live The Graph provider and document the fallback.

Phase 2: Implement the live Graph client

Time budget: 2.5 hours.

Tasks

1. Create `packages/graph-client` .
2. Implement `queryAddressActivity(address)` .
3. Validate the address before querying.
4. Send the smallest query that returns useful activity fields.
5. Normalize the provider response into `NormalizedEvidence` .
6. Handle empty results and incomplete results.
7. Add a timeout of 10 seconds.
8. Do not return raw provider errors to the browser.

Acceptance criteria

- A valid address returns normalized evidence.
- An empty address returns zero counts without crashing.
- An invalid address returns HTTP 400.
- Provider timeout returns a controlled error.
- The response identifies `The Graph` as the data source.
- The project can run with a real provider and not only static data.

Antigravity instruction

Keep the Graph client independent from Express and the frontend. Write one unit test for valid data, one for empty data, and one for provider failure. Do not add support for multiple chains.

Phase 3: Implement risk scoring and report generation

Time budget: 2 hours.

Tasks

1. Create `packages/risk-engine` .
2. Implement the transparent scoring rules.
3. Create the `RiskLabel` union type.
4. Create `packages/report-generator` .
5. Add the constrained AI prompt.
6. Add JSON validation.

7. Add deterministic fallback output.
8. Include limitations in every report.

Acceptance criteria

- The same evidence produces the same deterministic label.
- Incomplete evidence always produces `Needs review` .
- The LLM cannot change the deterministic label.
- Malformed LLM output falls back to a valid report.
- No report uses the words `guaranteed safe` , `scam confirmed` , or `financial advice` .

Antigravity instruction

The AI explains the evidence but does not control the core score. Keep scoring deterministic, auditable, and easy to show to judges.

Phase 4: Implement the x402-gated report API

Time budget: 3 hours.

Tasks

1. Create `apps/report-service` .
2. Implement `POST /v1/safety-report` .
3. Return HTTP 402 when the payment header is missing.
4. Configure the service as a Hedera testnet x402 resource server.
5. After payment verification, call the Graph client and report generator.
6. Return the report JSON.
7. Add request IDs and structured logs.
8. Add a health endpoint: `GET /health` .
9. Add a safe CORS configuration for local development.

Acceptance criteria

- `GET /health` returns `{ "status": "ok", "network": "testnet" }` .
- An unpaid report request returns HTTP 402.
- A paid request returns HTTP 200 and a valid report.

- The report contains live Graph evidence.
- The service cannot start in mainnet mode.
- Logs clearly show payment required, payment verified, Graph query, and report generated.

Antigravity instruction

Reuse a known Hedera x402 example or official SDK pattern. Do not implement x402 cryptography manually. Keep the payment amount fixed and small. Never log private keys or payment signatures.

Phase 5: Implement the agent payment client

Time budget: 2 hours.

Tasks

1. Create `apps/agent` .
2. Implement a request to the report API without payment.
3. Parse the HTTP 402 response.
4. Construct and sign the Hedera testnet payment using the dedicated payer account.
5. Retry the request with the correct x402 header.
6. Return payment lifecycle steps to the frontend.
7. Add `scripts/test-paid-request.ts` .

Acceptance criteria

The following command must work:

Bash

```
npm run test:paid-request -- --address 0xYOUR_TESTNET_ADDRESS
```

The output must show:

Plain Text

```
payment_required  
payment_signed  
payment_submitted
```

```
payment_verified  
report_generated
```

The command must return a valid report or a clearly explained provider error. It must not silently fall back to a fake paid response.

Day 2 — Interface, testing, documentation, and submission

Phase 6: Build the one-page web interface

Time budget: 3 hours.

Required interface components

1. Header with project name and track technologies.
2. Testnet-only status badge.
3. Address input.
4. Subject type selector with `Wallet` and `Protocol`.
5. Demo address button.
6. Generate report button.
7. Payment progress timeline.
8. Risk label card.
9. Summary paragraph.
10. Evidence table.
11. Reasons list.
12. Recommended checks list.
13. Limitations and disclaimer.
14. Testnet payment evidence.
15. Error message area.

Visual requirements

- Keep the interface clean and professional.
- Use a dark or neutral security-oriented design.

- Use green, yellow, and red only for labels, not as a claim of certainty.
- Use the label `High concern` , not `Malicious` .
- Make the testnet status obvious.
- Do not add a dashboard, charts, account system, or settings page.

Acceptance criteria

A judge can open the application and understand the following without reading the README:

- What problem the application solves.
- That The Graph supplies live data.
- That Hedera testnet x402 is used for payment.
- That the report is limited and educational.
- Whether the payment and report succeeded.

Phase 7: Full integration testing

Time budget: 2.5 hours.

Test cases

Test	Expected result
Valid testnet address	Paid report is generated
Demo address button	Address is populated and report can run
Invalid address	HTTP 400 and friendly message
Empty address	Frontend prevents submission
Missing payment	API returns HTTP 402
Insufficient payer balance	Friendly payment failure
Graph timeout	Controlled provider error
Empty Graph result	<code>Needs review</code> report
Incomplete Graph result	<code>Needs review</code> report with limitation
LLM unavailable	Deterministic fallback report
Mainnet environment value	Application refuses to start

Private key accidentally logged

Must not occur

Acceptance criteria

Run all tests from a clean terminal. At least one test must be a real paid testnet request. Do not rely only on mocked payment tests.

Phase 8: Prepare README and hackathon proof

Time budget: 2 hours.

The README must contain these sections in this order:

1. Project name and one-sentence description.
2. Problem.
3. Solution.
4. Live demo or local setup instructions.
5. Architecture diagram.
6. The Graph integration.
7. Hedera x402 integration.
8. End-to-end payment flow.
9. Risk-scoring method.
10. Environment variables.
11. How to run locally.
12. How to reproduce a paid request.
13. Security and testnet limitations.
14. Track qualification mapping.
15. Known limitations.
16. License.

Include commands such as:

Bash

```
npm install
cp .env.example .env
npm run check:providers
```

```
npm run dev
npm run test:paid-request -- --address 0xYOUR_TESTNET_ADDRESS
```

The README must clearly state:

This project uses Hedera testnet only. No mainnet funds or mainnet credentials are required or supported.

Phase 9: Record the demo video

Time budget: 1.5 hours.

Keep the video between two and four minutes.

Demo sequence

1. Explain the problem in 15 seconds.
2. Show the application and testnet badge.
3. Enter the demo address.
4. Click Generate Report.
5. Show HTTP 402 or the payment-required state.
6. Show the signed and submitted Hedera testnet payment.
7. Show the live Graph evidence.
8. Show the generated report.
9. Show the payment or transaction evidence.
10. Explain the two track integrations.
11. State the limitations honestly.

Demo narration

Use this explanation:

ChainGuard Lite helps a crypto user interpret unfamiliar wallet activity before interacting with it. The report is powered by live indexed blockchain data from The Graph. The report endpoint is not free: the agent receives HTTP 402, pays a small amount of testnet HBAR through Hedera x402, and then receives the report. The score is transparent, the AI only explains the evidence, and the product does not claim to guarantee safety.

11. Final acceptance checklist

The project is complete only when every item below is true.

Product

- ☐ One-page interface works.
- ☐ User can enter a testnet address.
- ☐ User can generate a report.
- ☐ Payment progress is visible.
- ☐ Report includes risk label, reasons, evidence, checks, and limitations.
- ☐ Testnet-only disclaimer is visible.

The Graph

- ☐ Live Graph provider request works.
- ☐ Raw data is normalized.
- ☐ Evidence comes from the live query.
- ☐ README shows the query or MCP request.
- ☐ The AI performs reasoning over Graph-derived evidence.

Hedera

- ☐ API returns HTTP 402 without payment.
- ☐ Agent signs a real testnet payment.
- ☐ Facilitator verifies or settles the payment.
- ☐ Report is returned after payment processing.
- ☐ No mainnet code path is required.
- ☐ Payment evidence is available for the video.

Reliability

- ☐ Invalid address is handled.
- ☐ Provider timeout is handled.

- ☐ Empty data is handled.
- ☐ LLM failure has a fallback.
- ☐ Mainnet configuration is rejected.
- ☐ Secrets are not committed or logged.

Submission

- ☐ Public GitHub repository.
 - ☐ Complete README.
 - ☐ Architecture diagram.
 - ☐ Setup instructions.
 - ☐ Reproducible paid-request command.
 - ☐ Two-to-four-minute demo video.
 - ☐ Track mapping included.
 - ☐ No fake payment or fake live-data claims.
-

12. Antigravity operating instructions

Use the following rules while building:

1. Work phase by phase.
2. At the end of every phase, run the phase acceptance tests.
3. Do not begin UI polish while the paid API flow is broken.
4. Do not add optional features until the required tracks work.
5. If a provider integration fails for more than 30 minutes, use the documented fallback.
6. Do not replace live Graph data with static data in the final demo.
7. Do not replace the real Hedera payment with a fake success message.
8. Keep every integration small and visible.
9. Prefer a working simple implementation over an ambitious incomplete implementation.
10. Keep a `BUILD_LOG.md` documenting commands run, errors encountered, and final fixes.

At the end of each phase, report:

Plain Text

Phase: <number and name>

Completed:

- ...

Verification commands:

- ...

Results:

- ...

Known issues:

- ...

Next phase:

- ...

13. Final implementation priority

If time becomes limited, implement in this order:

1. Real Hedera testnet x402 payment.
2. Real The Graph live query.
3. Deterministic evidence and risk scoring.
4. Report JSON response.
5. Simple browser interface.
6. AI explanation.
7. Documentation and demo video.
8. Optional Bazantic integration.

If only six hours remain, remove the optional AI provider, remove Bazantic, and use the deterministic report generator. Do not remove the real Graph query or the real Hedera payment because those are the two core track requirements.

14. Final project statement

ChainGuard Lite is a testnet-only, pay-per-report crypto safety assistant. It uses The Graph to retrieve live indexed blockchain activity, applies transparent screening rules,

and uses AI to explain the evidence. An agent pays for each report using Hedera testnet x402, demonstrating a complete machine-to-service payment workflow without mainnet risk or unnecessary protocol complexity.

References

- [1] The Graph Subgraph MCP Introduction
 - [2] Hedera and the x402 payment standard
 - [3] Hedera x402 inference pay-per-request proof of concept
 - [4] x402 Quickstart for Buyers
 - [5] Blocky402 x402 facilitator
 - [6] ETHGlobal ETHOnline 2026 prize tracks
 - [7] The Graph Standardized Subgraphs
-

Implementation assumption: AntigraVity must validate the exact available testnet Subgraph and x402 facilitator configuration during Phase 1. If the Subgraph MCP is difficult to connect, use a direct live GraphQL request to a The Graph provider. If the selected payment asset requires extra setup, use the simplest facilitator-supported testnet asset and document the choice.